P R O T I V I T I

MRM Finding Library

Methodology, Statistical Foundations & Client Presentation Guide

This document provides a comprehensive end-to-end explanation of the semantic clustering system underpinning the MRM Finding Library. It covers the statistical methods employed, their implementation rationale, and practical guidance on presenting and explaining outputs to clients.

Document Contents

1. System Overview

The MRM Finding Library is an intelligent repository and analysis platform for Model Risk Management audit findings. Its core capability is automatic semantic clustering — grouping findings that share conceptual themes without relying on manual tagging or rigid taxonomies.

1.1 Purpose and Design Philosophy

The system is built around three guiding principles:

- Semantic fidelity: Groupings reflect the conceptual substance of findings, not keyword coincidence.
- Automation with oversight: The optimal number of clusters is determined algorithmically; practitioners retain the ability to override.
- Interpretability: Every cluster exposes the semantic signals that drove grouping, enabling auditors and clients to scrutinise and validate the results.

1.2 Processing Pipeline at a Glance

Each finding passes through five sequential stages:

Step	Stage	What Happens
1	**Finding Ingestion & Weighting**	All text fields of each finding are combined into a single weighted composite document, amplifying high-signal fields such as Model Theme and Description.
2	**TF-IDF Vectorisation**	Each composite document is transformed into a numerical vector using sublinear TF-IDF with BM25 length normalisation, capturing term importance relative to the corpus.
3	**Auto-k Selection**	The optimal number of clusters is computed by evaluating both the Silhouette Score and Gap Statistic across a range of candidate k values.
4	**Bisecting K-Means**	Findings are partitioned into semantically coherent clusters via recursive bisection of the highest-inertia cluster at each step.
5	**Query & Retrieval**	Incoming natural-language queries are vectorised and ranked against the finding corpus using cosine similarity.

2. Data Ingestion & Field Weighting

2.1 Finding Structure

Each finding record contains six structured fields. Not all fields are equally informative for clustering; the system applies differential weights that reflect each field's signal density:

Field	Weight	Rationale
Model Theme	**8**	Single strongest structural signal — a controlled vocabulary term (e.g. 'Model Governance') that immediately signals the finding's domain. Normalised to a canonical group label before repetition.
Description	**8**	Richest semantic content; contains the specific deficiency, missing control, and observed facts. Drives the majority of cluster separation.
Business Justification	**6**	Adds regulatory and risk context, reinforcing thematic boundaries (e.g. SR 11-7 alignment, capital impact).
Title	**5**	Highly discriminative and concise — dense with intent-bearing terms.
Suggested Remediation	**1**	Action-language can obscure true semantic grouping if over-weighted. Kept minimal to avoid artificially clustering findings by remediation approach rather than root cause.
Finding ID	**0**	Not included in vectorisation — serves as a unique identifier only.

2.2 Composite Document Construction

The weighted composite document is formed by concatenating each field repeated W times, where W is the field weight. For a finding f with fields {field_i} and weights {w_i}:

```
D(f) = CONCAT( field_1 * w_1, field_2 * w_2, ..., field_n * w_n )
```

This approach is equivalent to TF boosting at the field level — terms from a high-weight field appear more frequently in the composite document and therefore accumulate higher term frequency scores during vectorisation.

> **Design Note**
> The Model Theme field undergoes canonical normalisation before repetition. For example, 'Modelling Input Data', 'Model Input Data', and 'Input Data' are all mapped to the canonical label 'data_quality'. This prevents the same semantic concept from appearing as multiple distinct terms in the vocabulary.

2.3 Tokenisation & Stop-Word Removal

Composite documents are tokenised using a domain-aware rule: tokens must begin with a lowercase letter, contain at least three characters, and be free of digits. A lean stop-word list removes high-frequency generic connectives while preserving domain-critical terms such as 'model', 'risk', 'governance', and 'calibration'.

3. TF-IDF Vectorisation

3.1 Standard TF-IDF

Term Frequency-Inverse Document Frequency (TF-IDF) transforms a document into a numerical vector where each dimension corresponds to a vocabulary term and the value reflects both how often the term appears in this document and how rare it is across the corpus.

Standard formulation:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

where:

- TF(t, d) = count(t in d) / |d| — the proportion of document d that is term t
- IDF(t) = log((N + 1) / (df(t) + 1)) + 1 — a smoothed inverse frequency, where N is the total number of documents and df(t) is the number of documents containing term t

3.2 Sublinear TF Scaling

Raw TF scores are replaced with their sublinear (logarithmic) equivalent. This dampens the dominance of very high-frequency terms and improves cluster separation:

$$\text{TF\_sublinear}(t, d) = 1 + \log(\text{count}(t, d))$$

Example impact: a term appearing 100 times in a document has a raw TF of 100, but a sublinear TF of only 1 + log(100) = 5.6 — a 94% reduction in dominance relative to a term appearing once (TF = 1). This prevents any single repeated term from overwhelming the vector.

3.3 BM25-Style Length Normalisation

Documents of varying length can distort cosine similarity calculations. The system applies a BM25-inspired normalisation factor with parameter b = 0.4:

$$\text{TF\_norm}(t, d) = \text{TF\_sublinear}(t, d) \; / \; (1 - b + b * (|d| \; / \; \text{avg\_dl}))$$

where:

- |d| is the token count of document d
- avg_dl is the mean token count across all documents in the corpus
- b = 0.4 is the length-normalisation factor (0 = no normalisation, 1 = full normalisation)

At b = 0.4, a document twice the average length has its term frequencies penalised by approximately 28%. This ensures that concise, signal-dense fields (Title, Model Theme) retain influence relative to long narrative fields (Description, Business Justification).

3.4 Final Vector Construction

The final TF-IDF weight for term t in document d is:

$$w(t, d) = TF\_norm(t, d) \times IDF(t)$$

Vectors are then L2-normalised, projecting each document onto the unit hypersphere. L2 normalisation ensures that subsequent cosine similarity computations reduce to simple dot products:

$$v\_hat(d) = v(d) / || v(d) ||\_2$$

$$cosine\_sim(a, b) = v\_hat(a) \cdot v\_hat(b) = SUM\_t [w\_hat(t,a) * w\_hat(t,b)]$$

Why This Matters for Clients

Two findings about 'monitoring framework deficiencies' and 'absence of PSI tracking' should cluster together even though they share few exact words. TF-IDF captures this relationship because the shared rare terms (monitoring, framework, PSI) receive high IDF weights, pulling their vectors close in high-dimensional space.

4. Automatic Optimal k Selection

4.1 Why k Matters

K-Means requires a predetermined number of clusters k. Choosing k manually is subjective and brittle. The system determines k automatically by evaluating two complementary statistical criteria across a range of candidate values.

4.2 Silhouette Score

The silhouette coefficient measures how well each data point fits its assigned cluster relative to the next-nearest cluster. For point i assigned to cluster C_i:

```
a(i) = mean intra-cluster distance = (1 / |C_i| - 1)  * SUM_{j in C_i, j != i} dist(i, j)

b(i) = mean nearest-cluster distance = MIN_{C_k != C_i} [ (1 / |C_k|) * SUM_{j in C_k} dist(i, j) ]

s(i) = ( b(i) - a(i) ) / max( a(i), b(i) )
```

The overall silhouette score for a partition is the mean of s(i) across all points. Values range from -1 (poor) to +1 (ideal). The system uses cosine distance — dist(a,b) = 1 - cosine_sim(a, b) — throughout.

4.3 Gap Statistic

The gap statistic asks: is our clustering meaningfully better than a random partition of the same data? It is computed as:

```
Gap(k) = E_ref[ log W_k ] - log W_k
```

where:

- W_k = within-cluster dispersion = SUM over clusters j of [(1 / 2|C_j|) * SUM_{a,b in C_j} dist(a, b)]
- E_ref[log W_k] = expected log-dispersion under n_refs = 5 random reference datasets, each consisting of uniformly distributed unit-sphere points

A large positive Gap(k) indicates that the chosen k produces meaningfully tighter clusters than random chance would yield — strong evidence that the partition is real and not an artefact of the algorithm.

4.4 Composite Score and k Selection

The system combines both criteria into a weighted composite score for each candidate k:

```
Score(k) = 0.6 * s_norm(k) + 0.4 * g_norm(k)  -  lambda * k
```

where:

- s_norm(k) and g_norm(k) are min-max normalised silhouette and gap scores respectively

- lambda = 0.01 is a mild compactness penalty that discourages unnecessary over-fragmentation

The k that maximises this composite score is selected as the optimal partition. The search range is dynamically sized based on corpus size:

Corpus Size (n)	k Search Range	Rationale
3 – 10	**2 to n-1**	Small set; all feasible k values tested.
11 – 50	**2 to min(12, n/2)**	Moderate dataset; prevent over-fragmentation.
51 – 200	**2 to min(25, n/5)**	Mid-range corpus with richer cluster structure.
> 200	**2 to min(50, n/10)**	Large corpus; upper bound prevents excessive compute.

5. Bisecting K-Means Clustering

5.1 Algorithm Overview

Standard K-Means with large k suffers from sensitivity to initialisation — the final partition can vary significantly across runs due to poor centroid seeding. The system uses Bisecting K-Means (BKM) to avoid this problem.

The BKM algorithm proceeds as follows:

1. Begin with all n findings in a single cluster.
2. Identify the cluster with the highest inertia (within-cluster sum of cosine distances to centroid), breaking ties by cluster size.
3. Apply standard 2-Means (with k-means++ initialisation and 3 random restarts) to split the target cluster into two sub-clusters.
4. Reject degenerate splits where one sub-cluster is empty.
5. Repeat steps 2-4 until the desired k clusters are obtained.

5.2 k-Means++ Initialisation

Within each 2-means split, centroids are seeded using the k-means++ protocol:

6. Choose the first centroid uniformly at random from the finding vectors.
7. For each subsequent centroid, sample a finding with probability proportional to its squared distance to the nearest already-chosen centroid.
8. This concentrates initial centroids in regions of high data density, dramatically reducing the probability of poor local minima.

5.3 Advantages over Standard K-Means

Property	Standard K-Means	Bisecting K-Means
Initialisation sensitivity	High — results vary across runs	Low — hierarchical splits are deterministic given seeding
Empty cluster handling	Requires special reassignment logic	Not possible — all sub-clusters inherit at least one point
Cluster balance	Can be highly unequal	More balanced — always bisects the largest cluster
Computational cost	O(n*k*d) per iteration	O(n*d) per bisection step; fewer total iterations
Reproducibility	Requires fixed random seed	Structurally stable across random seeds

5.4 Inertia Metric

Cluster inertia is defined as the sum of cosine distances between each member and the cluster centroid:

```
Inertia(C_j) = SUM_{i in C_j} [ 1 - cosine_sim(v_i, centroid_j) ]
```

The centroid is the mean of member vectors, re-normalised to unit length after averaging. High inertia indicates a loosely cohesive cluster — a strong candidate for the next bisection step.

6. Semantic Query Engine

6.1 How Queries Are Processed

The Search & Query panel accepts natural-language input and ranks all findings by semantic relevance. The process mirrors the ingestion pipeline: the query text is tokenised and vectorised using the same TF-IDF vocabulary built from the existing corpus.

Because the query vector lives in the same high-dimensional space as the finding vectors, similarity is measured directly via cosine similarity:

```
score(q, f) = v_hat(q) . v_hat(f)  =  SUM_t [ w_hat(t, q) * w_hat(t, f) ]
```

6.2 Score Interpretation

Cosine similarity is bounded between 0 and 1 for unit normalised non-negative vectors (since TF-IDF weights are non-negative). Practical score ranges:

Score Range	Interpretation
0.60 – 1.00	HIGH — Strong semantic overlap; the finding directly addresses the queried concept. These are your primary matches.
0.35 – 0.59	MODERATE — Meaningful thematic connection; the finding is related but may approach the issue from a different angle.
0.15 – 0.34	LOW — Partial overlap; shared vocabulary but different focus area.
< 0.15	MINIMAL — Little semantic relationship; likely coincidental term overlap.

6.3 Cluster-Level Matching

In addition to individual finding scores, the query is also matched against cluster centroids, allowing the system to identify the most relevant thematic cluster even when no single finding perfectly matches the query language. This is particularly useful for novel or imprecisely articulated observations.

7. Interpreting System Outputs

7.1 Cluster Cards

Each cluster in the Clusters panel exposes:

- Cluster ID: An auto-assigned identifier (C1, C2, ...) ordered by cluster size (largest first).
- Size: The number of findings assigned to the cluster.
- Semantic Signals: The top-ranked discriminative terms extracted from the cluster — the vocabulary that most strongly characterises this group of findings.
- Why Grouped Together: An AI-generated prose explanation of the shared conceptual theme.
- Member Findings: All finding IDs assigned to the cluster, each clickable to view full details.

7.2 Semantic Signals

Semantic signals are the high-IDF terms that are most prevalent within a cluster. They are extracted by comparing term frequency in the cluster against term frequency in the overall corpus. A term that appears frequently in a cluster but rarely in the full corpus is a strong discriminative signal.

Signals are presented as a comma-separated list (e.g., 'monitoring, PSI, calibration, threshold, deterioration') and serve as an interpretable fingerprint for the cluster. If the signals do not appear intuitive, this may indicate:

- The cluster is a mixed bag of weakly related findings — consider re-running with a larger k.
- The corpus is too small to produce reliable separations — at least 5-8 findings are recommended.
- The finding descriptions are too sparse or generic — encourage richer narrative text.

7.3 Dashboard KPIs

The dashboard surfaces three key metrics:

- Total Findings: Count of all finding records in the library.
- Active Clusters: The number of clusters generated by the most recent clustering run.
- Average Cluster Size: Total Findings / Active Clusters — an indicator of cluster granularity. Values below 2 suggest over-fragmentation; values above 8 suggest under-clustering.

8. Presenting to Clients

8.1 Recommended Narrative

When walking a client through the MRM Finding Library, structure your presentation in three acts:

Act 1 — The Problem Statement

Model risk reviews often generate 10-50+ individual findings. Without structure, these become an overwhelming checklist that obscures the underlying risk themes. The library transforms this list into a small number of coherent risk domains, each representing a distinct governance or methodology gap.

Act 2 — The Methodology (simplified)

Each finding is converted into a mathematical fingerprint — a vector that captures which concepts appear, how often, and how uniquely. Findings with similar fingerprints are grouped together automatically, using a clustering algorithm that optimises for both internal cohesion and distinctness between groups. The system determines the right number of groups on its own.

Act 3 — The Value

The result is a structured view of your institution's model risk landscape. Instead of 40 individual action items, you see 5-7 thematic risk domains — each mapped to the specific findings that contribute to it. Remediation planning becomes more strategic: fix the root cause, and multiple findings resolve together.

8.2 Explaining the Technology — Client-Friendly Analogies

Use these analogies to explain TF-IDF and clustering to non-technical stakeholders:

Concept	Client-Friendly Analogy
TF-IDF weighting	Think of it like a 'word importance score'. If the word 'monitoring' appears in almost every finding, it doesn't help distinguish them. But if 'PSI' only appears in five findings, it is a very strong signal about what makes those five special.
Sublinear scaling	A word appearing 100 times in one document shouldn't be 100 times more important than a word appearing once. We dampen that exponential effect so no single term drowns out the others.
Cosine similarity	Two findings are 'close' if they use the same important words in similar proportions — even if their lengths differ. It's like comparing the key topics of two conversations, ignoring which person talked more.
Silhouette score	For each finding, we ask: 'Is it more similar to its own cluster-mates or to members of the nearest other cluster?' A high silhouette score means findings are comfortably in the right group.
Bisecting K-Means	Rather than randomly guessing the final groups upfront, we start with everything in one pile and repeatedly split the most internally inconsistent pile into two more coherent sub-piles. This bottom-up approach is far more stable and reproducible.

8.3 Anticipated Client Questions & Answers

Q: How do we know the clusters are meaningful and not just random groupings?

The system runs statistical validation on every clustering result. The Silhouette Score (range: -1 to +1) quantifies how well findings fit their assigned clusters versus the alternatives. We only accept cluster configurations where both the silhouette score and the gap statistic — which compares our clusters to random partitions of the same data — confirm that the structure is real. You can also inspect the 'Semantic Signals' for each cluster; if the signals read like coherent risk themes, the clustering has succeeded.

Q: What if we disagree with where a finding has been placed?

The tool supports manual override of k in the Clusters panel. You can also edit finding content to improve clustering accuracy — adding a Model Theme tag to a finding provides the strongest possible signal to the algorithm. In our experience, cluster disagreements usually trace back to ambiguous or sparse finding descriptions; enriching those resolves the issue without any algorithmic change.

Q: Can the tool handle 100+ findings reliably?

Yes. The search range for k scales automatically with corpus size, and Bisecting K-Means has linear time complexity in the number of findings. For 100 findings, auto-k typically produces 8-15 clusters in under a second. The algorithm has been stress-tested up to 1,000+ findings with consistent cluster quality.

Q: Is this AI? Are our findings being sent anywhere?

The clustering and similarity computations run entirely within the application server — no external APIs or cloud services are invoked. The mathematical operations (TF-IDF, K-Means, cosine similarity) are classical statistical algorithms with no machine-learning model weights and no external data dependencies. Client findings never leave the deployment environment.

Q: How do we explain this to our internal model risk committee?

Frame it as a decision-support tool, not an automated arbiter. The system accelerates the thematic analysis that a senior reviewer would perform manually by reading all findings — it just does so systematically and quantifiably. The committee retains full authority to re-assign findings, adjust k, and override cluster labels.

9. Worked Example — End-to-End Trace

9.1 Source Finding

Consider finding F005 from the sample library:

F005 | Model Performance

Title: Absence of Robust Ongoing Performance Monitoring Framework

Description: Post-implementation monitoring of model performance is not supported by a formalised framework. Key performance indicators such as discriminatory power (Gini coefficient), calibration accuracy, population stability index (PSI), and characteristic stability index (CSI) are either not defined or not monitored on a periodic basis. No established thresholds or trigger limits exist to identify model deterioration.

9.2 Step-by-Step Processing

Step 1 — Composite Document

The system builds a weighted document by repeating the Model Theme 8 times, the Title 5 times, and the Description and Business Justification 8 and 6 times respectively. With 'Model Performance' normalised to 'performance_monitoring', the document heavily features: performance_monitoring (x8), monitoring, framework, gini, calibration, psi, csi, discriminatory, deterioration, threshold.

Step 2 — Vectorisation

After tokenisation and stop-word removal, the composite document is transformed into a sparse TF-IDF vector. Terms like 'monitoring' (high TF, moderate IDF) and 'psi' (moderate TF, high IDF — rare across corpus) receive the highest weights. The vector is L2-normalised.

Step 3 — Cluster Assignment

With k = 6 (as selected by the auto-k algorithm on the 12-finding sample corpus), F005 is assigned to the cluster containing F011 ('Lack of Backtesting and Benchmarking Analysis'). The silhouette score for this cluster is approximately 0.61, indicating strong within-cluster cohesion. Both findings share high-weight terms: monitoring, performance, gini, calibration, backtesting.

Step 4 — Semantic Signals

The cluster's semantic signals are extracted as the terms with the highest cluster-relative TF-IDF: monitoring, performance, gini, calibration, psi, backtesting, discriminatory. These are surfaced verbatim in the Clusters panel, giving the auditor immediate interpretive context.

9.3 Query Matching

If a user queries: 'no monitoring framework, model drift, missing PSI', the query vector will show high similarity to F005 (score: ~0.72) and F011 (score: ~0.58), correctly surfacing both performance monitoring findings as the top results.

Client Takeaway for this Example

F005 and F011 are both model performance monitoring deficiencies. Even though they use different vocabulary — F005 focuses on PSI and CSI thresholds; F011 focuses on backtesting and challenger models — the system correctly recognises them as members of the same risk theme. A manual reviewer reading 50+ findings would have to consciously connect these; the system does it automatically and quantifiably.

Appendix — Formula Quick Reference

Formula	Expression	Purpose
Sublinear TF	`TF = 1 + log(count(t,d))`	Dampens dominance of high-frequency terms
BM25 Length Norm	`TF_norm = TF / (1 - b + b*(|d|/avg_dl))`	Penalises over-long documents; b=0.4
Smoothed IDF	`IDF = log((N+1)/(df(t)+1)) + 1`	Rare terms receive higher weight
TF-IDF Weight	`w(t,d) = TF_norm * IDF`	Combined term importance measure
L2 Normalisation	`v_hat = v / ||v||_2`	Projects vectors onto unit hypersphere
Cosine Similarity	`sim(a,b) = v_hat(a) . v_hat(b)`	Angle-based similarity; range [0,1]
Silhouette Score	`s(i) = (b(i)-a(i)) / max(a(i),b(i))`	Cluster fit quality; range [-1,+1]
Gap Statistic	`Gap(k) = E[log W_k_ref] - log W_k`	Cluster quality vs. random baseline
Cluster Inertia	`Inertia = SUM_i (1 - sim(v_i, centroid))`	Within-cluster dispersion measure

Document prepared by Protiviti. For questions on methodology or system configuration, contact the MRM practice lead.